

Übungsblatt 6

Ausgabe: 15.01.2024 **Abgabe:** 26.01.2024, 12:00 Uhr

Hausübung

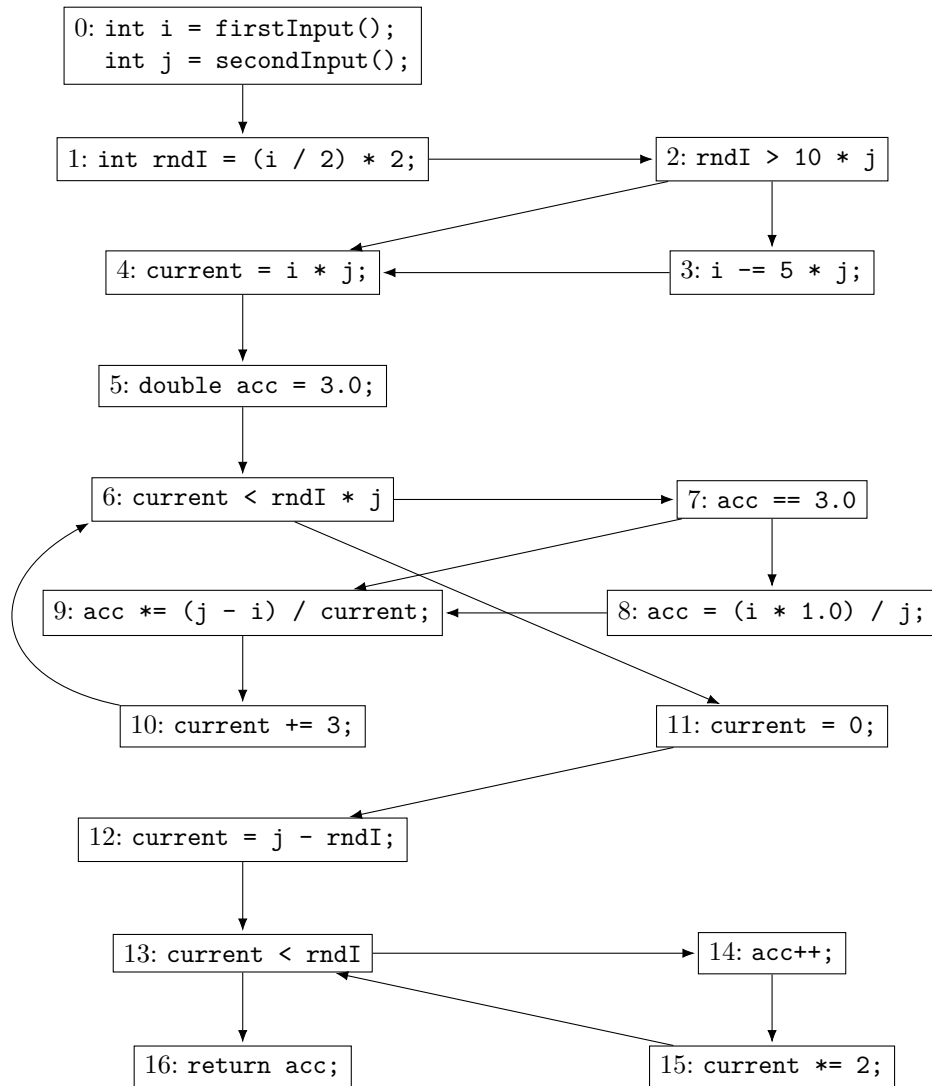
Aufgabe 6.1 (Datenflussanalyse)

8 P.

Betrachten Sie den Kontrollflussgraphen auf der nächsten Seite. Jede Zuweisung ist eindeutig durch die Nummer ihres Knotens identifiziert. Auf diesem Graphen sollen jetzt eine **Reaching Definitions-** und eine **Live-Variables-Analyse** durchgeführt werden. Erstere ist aus der Vorlesung bekannt. Die Eingabeparameter werden nicht von dieser Analyse erfasst.

Eine Live-Variables-Analyse untersucht, welche Variablen in einer Zeile lebendig sind, d. h. ihr Wert wird später im Programm noch benutzt. Wir gehen ähnlich wie bei der Reaching-Definitions-Analyse vor, betrachten jedoch nur Mengen von Variablen. Im letzten Knoten sind *keine* Variablen lebendig. Ein Knoten tötet Variablen, denen ein Wert zugewiesen wird und generiert Variablen, die ausgelesen werden (auch in einem -Statement). Variablen, die geschrieben *und* gelesen werden, werden keiner der Mengen hinzugefügt. Die Menge lebendiger Variablen in einem Knoten bestimmt sich aus der Vereinigung lebendiger Variablen aller Nachfolger sowie der lokal generierten Variablen, minus der lokal getöteten Variablen.

- (a) Bestimmen Sie die Kill-Menge für eine Reaching Definitions-Analyse für jeden Knoten im Kontrollflussgraphen. 1 P.
- (b) Bestimmen Sie ebenfalls die Gen-Menge für eine Reaching Definitions-Analyse für jeden Knoten. 1 P.
- (c) Führen Sie die Reaching Definitions-Analyse durch und berechnen Sie die erreichenden Definitionen für jeden Knoten (RD_{exit} nach dem letzten Schritt). 2 P.
- (d) Bestimmen Sie die Kill-Menge für eine Live-Variables-Analyse für jeden Knoten im Kontrollflussgraphen. 1 P.
- (e) Bestimmen Sie ebenfalls die Gen-Menge für eine Live-Variables-Analyse für jeden Knoten. 1 P.
- (f) Führen Sie die Live-Variables-Analyse durch und berechnen Sie die lebendigen Variablen für jeden Knoten. 2 P.



Aufgabe 6.2 (Strukturelle Operationelle Semantik)

6 P.

Nutzen Sie die SOS-Semantik, um folgendes Programm in erweiterter While-Syntax auszuwerten:

```
[i := 42]1;  
[j := 0]2;  
do  
| [i := i * 2]3;  
| [j := j + 1]4;  
while [i < 100]5
```

Dazu müssen Sie allerdings die Semantik erweitern, um die zusätzlichen Programmkonstrukte auswerten zu können.

- (a) Definieren Sie eine oder mehrere Auswertungsregeln, um **do...while...** korrekt auszuwerten. Das Statement soll sich wie ein **while...do...** verhalten, aber erst nach dem Block die Bedingung überprüfen. 1 P.
- (b) Benutzen Sie die Auswertungsregeln der SOS und Ihre Erweiterung, um das Programm auszuwerten. Die Auswertung von Rechenausdrücken müssen Sie nicht aufschlüsseln. Sie können im Interesse der Lesbarkeit Ausdrücke durch ihr Label ersetzen. 5 P.

Beispiel: Wir werten $[x := 1]^1$ **if** $[x = 1]^2$ **then** $[x := 0]^3$ **else** $[x := -x]^4$ aus:

Wir benötigen die folgenden Nebenrechnungen:

$$\begin{array}{c} \langle 1, \emptyset \rangle \\ \xRightarrow{[ASS]} \{x \mapsto 1\} \end{array} \quad (1)$$

$$\begin{array}{c} \langle \text{if } 2 \text{ then } 3 \text{ else } 4, \{x \mapsto 1\} \rangle \\ \xRightarrow{[IF_T]} \langle 3, \{x \mapsto 1\} \rangle \end{array} \quad (2)$$

$$(3)$$

Jetzt können wir ableiten:

$$\begin{array}{c} \langle 1; \text{if } 2 \text{ then } 3 \text{ else } 4, \emptyset \rangle \\ \xRightarrow{[COMP_2], (1)} \langle \text{if } 2 \text{ then } 3 \text{ else } 4, \{x \mapsto 1\} \rangle \\ \xRightarrow{[COMP_1], (2)} \langle 3, \{x \mapsto 1\} \rangle \\ \xRightarrow{[ASS]} \{x \mapsto 0\} \end{array}$$

Sie müssen also für jede Anwendung der Kompositionsregel eine Nebenrechnung für den ersten Teil ausführen¹.

¹Eigentlich würde man hier einen Beweisbaum notieren. Aus Gründen der Lesbarkeit ist eine Nebenrechnung mit einer Anmerkung am Schritt-Pfeil aber für diese Übung ausreichend.

Aufgabe 6.3 (Symbolische Ausführung)

6 P.

Sie wollen das folgende Programm auf Korrektheit untersuchen und benutzen dazu die Technik der symbolischen Ausführung, um Angaben über die Ergebnisse zu erhalten.

```
[y := 2x]1;  
if [y > x]2 then  
  [ y := (2 * x * x)/y]3;  
else  
  [ x := 1]4;  
[x := 2x]5;  
if [y < 0]6;  
  then  
    while [x = 1]7;  
    do  
      [ skip]8;  
  else  
    while [x < 0]9;  
    do  
      [ skip]10;  
[x := y]11;
```

- (a) Führen Sie das Programm symbolisch aus. Benutzen Sie dafür die aus der Vorlesung bekannten Regeln². Geben Sie bei jedem Schritt an, welche Regeln Sie verwenden. Wenn Sie in einem Schritt mit SAT oder UNSAT abbrechen, vermerken Sie dies. Geben sie für jeden SAT-Fall ein Modell an. 4 P.
- (b) Überprüfen Sie anhand Ihrer Ergebnisse, ob in jedem Zustand die Invariante $x|y \vee y|x$ (x teilt y oder y teilt x ; $\exists d \in \mathbb{Z} : dx = y \vee dy = x$) gilt. Falls ja, erläutern Sie in eigenen Worten, wie die symbolische Ausführung zur Überprüfung dieser Invariante genutzt werden kann. Falls nein, nennen Sie ein Gegenbeispiel. 1 P.
- (c) Entscheiden Sie jetzt, ob das Programm in jedem Fall terminiert. Begründen Sie Ihre Antwort. 1 P.

²Verwenden Sie hier auch die verkürzte Schreibweise aus der Aufgabe 6.2